

A. Objective Questions on Manual Testing

1. What is the primary purpose of manual testing?

- a. To automate the testing process
- b. To identify defects in software applications
- c. To analyze source code
- d. To write documentation

Answer: b

2. Which of the following is NOT a phase in the software testing life cycle?

- a. Test Planning
- b. Test Execution
- c. Test Deployment
- d. Test Closure

Answer : c

3. When writing a test case, which component is essential to include for clarity?

- a. Test Execution Date
- b. Pre-conditions
- c. User Stories
- d. Code Snippets

Answer : b

4. What does the term "test scenario" refer to?

- a. A detailed step-by-step process
- b. A high-level description of what to test
- c. A specific test case
- d. A report of defects

Answer: b

5. Which type of testing is performed to ensure that new changes do not adversely affect

existing features?

- a. Unit Testing
- b. Regression Testing
- c. Integration Testing
- d. Smoke Testing

Answer: b

6. What is the primary focus of User Acceptance Testing (UAT)?

- a. To test the application's performance under load
- b. To ensure the application meets the business requirements
- c. To verify code quality
- d. To conduct automated tests

Answer : b

7. Which of the following is a common challenge in manual testing?

- a. Lack of documentation
- b. High automation proficiency
- c. Predictable outcomes
- d. Limited testing tools

Answer: a

8. What is the best practice when a defect is found during testing?

- a. Ignore it if it seems minor
- b. Document it in a defect tracking system
- c. Fix it immediately without further testing
- d. Report it to the stakeholders without analysis

Answer: b

B. Scenario Based Question

a. When you are assigned to test a legacy system that has undergone minimal updates over the years. The documentation is outdated, and the system is critical for business operations. Describe your approach to testing this system. How would you gather information, determine test cases, and ensure that important functionalities are not overlooked?

Answer: Below is my point wise approach

- Discuss with support teams, long-term users, and product owners to understand critical workflows, pain points, and known failure areas.
- Review past incidents, defect history, change logs, and usage analytics to identify high-risk modules.
- Document current actual behavior, interfaces, dependencies, and data flows as baseline "living documentation."
- Classifying the features by business impact and likelihood of failure
- Creating scenario-based tests for happy paths, negative cases, permissions, and integrations.
- Creating a smoke suite and regression suite for core workflows, and running them for every change window.
- Performing walkthroughs of test coverage to confirm no critical process is missed.

b. When you work in an organization where the culture prioritizes speed and cost-cutting over thorough testing. You discover that this approach leads to frequent defects and customer dissatisfaction. As a QA professional, how do you balance your ethical responsibility to ensure quality with the organizational pressures to conform? What strategies would you employ to advocate for a change in mindset without jeopardizing your position?

Answer: Below is my point wise approach

- Make quality visible in business terms by connecting defects to customers, supporting cost, and rework effort.
- Tracking defect leakage, escaped defect severity, and release rollback frequency.
- Propose pragmatic improvements like risk based testing, better entry/exit criteria, mandatory smoke checks, and targeted automation for repetitive high-risk areas.

- If timelines are fixed, reduce lower priority test coverage explicitly while preserving critical path validation.

c. Why is it important to consider the end user's perspective in manual testing? How can adopting a user-centric approach to improve the testing process and the final product?

Answer: Importance of end user perspective in manual testing

- Users don't interact with internal components, they experience workflows, clarity, errors, and trust.

- User-centric testing improves.

- Usability testing improves, validates navigation, labels, messages, and accessibility.

- Catches issues in realistic data, devices, and usage patterns.

- Reduces frustration, support tickets, and abandonment.

- Test with real scenarios, varying skill levels, and context-driven exploratory sessions.

d. With the rise of automation in testing, what do you believe is the future role of manual testing? In what situations do you think manual testing is still essential, and why?

Answer:

- Manual testing remains essential where human judgment is required.

- Exploratory testing where discovering unknown risks and ambiguous behavior.

- UX/usability/accessibility checks, evaluating clarity, flow, and user confidence.

- When requirements are evolving and automation is unstable/expensive to maintain.

- Complex business logic validations.

- One-off or high variance scenarios, rare but high impact workflows not worth full automation.

- Automation handles repeatable regression but manual testing focuses on risk discovery, user value, and quality intelligence.

e. During a project, a serious defect is found that you believe could have been avoided with better communication among team members. However, blaming any individual could damage team morale and trust. How would you address the issue without pointing fingers? What steps would you take to foster a culture of accountability while ensuring that the team learns from this experience?

Answer:

- First run a postmortem focused on "what happened" and "why process failed," not "who failed."
- Build a clear timeline by adding requirement discussion, handoffs, implementation, testing, and release events.
- Identify process gaps by checking if there is unclear ownership, missing acceptance criteria, weak handoff, absent review checkpoints.
- Define corrective actions with owners and deadlines.
- Stronger definition of done
- Shared test strategy reviews
- Release readiness checklist
- Better defect triage communication
- Reinforce team trust by recognizing collective responsibility, celebrate improvements, and monitor whether changes reduce recurrence.

C. API Based Question

1. When designing an API, why is it important to consider the potential for future changes?

- a. To ensure that the API is always up-to-date
- b. To allow for backwards compatibility and minimize disruption for current users
- c. To reduce the complexity of the API
- d. To avoid using too many resources

Answer: b

2. As a QA you notice a sudden spike in API error rates, what should your first step be?

- a. Ignore it; it will likely resolve itself
- b. Investigate the cause of the spike by reviewing logs and monitoring performance
- c. Immediately notify all users of the outage
- d. Increase the server capacity without analysis

Answer: b

3. How would you evaluate the success of an API after its release?

- a. By the number of API requests made
- b. By user feedback and error rates

- c. By the amount of data processed
- d. By the speed of response times alone

Answer: b

4. What is a common drawback of using third-party APIs?
- a. They are usually free to use
 - b. They can introduce dependencies that may affect application functionality
 - c. They require less maintenance than in-house APIs
 - d. They are guaranteed to be secure

Answer : b

D. Automation Question

1. How would you approach testing a web application that dynamically loads content based on user interactions?
- a. Test only the initial load of the application
 - b. Use Playwright's built-in waiting mechanisms to ensure elements are loaded before interacting
 - c. Disable dynamic loading in the application for testing purposes
 - d. Ignore the dynamic content as it is not critical

Answer: b

2. As a QA when you encounter a test failure due to a timing issue, what strategies could you implement to resolve this?
- a. Reduce the number of tests to avoid timing issues
 - b. Increase the timeout settings or use explicit waits
 - c. Ignore the timing issues and move on
 - d. Run tests in parallel without any synchronization

Answer: b

3. In Playwright, how can you simulate user input for testing purposes?

- a. By using the `page.type()` method to enter text into input fields
- b. By directly manipulating the DOM using JavaScript
- c. By calling the `simulateUserInput()` function
- d. By using the `page.sendKeys()` method

Answer: a

4. When testing a web application that relies on third-party APIs, how can you ensure your tests are stable?

- a. Directly call the API in your tests
- b. Mock the API responses to control the testing environment
- c. Only test the UI without considering the API
- d. Use random data for testing

Answer: b

5. How would you organize your Playwright tests for a large application to maintain clarity and manageability?

- a. Keep all tests in a single file for simplicity
- b. Group tests by feature or functionality in separate files or folders
- c. Write tests in a random order to avoid confusion
- d. Use only one test case per file

Answer: b

6. When automating tests for a web application that uses AJAX calls, what is a key consideration?

- a. Ensure that all tests run without waiting for AJAX calls to complete
- b. Use Playwright's built-in waiting mechanisms to ensure that the application state is stable before proceeding with assertions
- c. Only test the initial page load and ignore AJAX functionality
- d. Test only the AJAX calls without validating the UI changes

Answer: b

E. Scenario Based API questions

1. Imagine as a QA you are responsible for maintaining a public API that has undergone several changes over the years. Some users are still relying on older versions, while new clients are using the latest version. What strategies would you implement to manage API versioning effectively? Consider how you would communicate changes to users and the potential impact on existing integrations.

Answer: I would use a clear versioning strategy (for example, URI versioning like /v1, /v2 or header-based versioning) and treat each published version as a contract. Older versions will remain supported for a defined period with a documented deprecation policy, timeline, and end date.

Communication approach will be by announcing changes early through multiple channels, sending remainder at milestones like announcement date, deprecation start etc. Provide support channels

2. Poorly documented APIs lead to developer frustration and increased support requests. If you were tasked with improving the documentation for an existing API, what key elements would you focus on? How would you ensure that the documentation is user-friendly and meets the needs of both novice and experienced developers?

Answer: Below are the key elements I would focus on:

- Clear getting-started guide with authentication, base URL, first successful request.
- Endpoint reference for every API with method, path, parameters, headers, request/response schemas, status codes, and error messages.
- Adding real examples.
- Error handling section that includes common errors, causes, and step-by-step fixes.
- Versioning and lifecycle that includes current versions, deprecated endpoints, migration notes.
- Security guidance like token handling, permissions, and best practices.
- Release notes with clear impact summaries.

And for User friendliness :

- Structure by use cases ("Create user", "Process payment") in addition to raw endpoint docs.
- Using consistent terminology and response formats across all pages.
- Adding interactive API like Postman collection for quick testing.
- Include troubleshooting and "known failures" sections from actual support tickets.